

A	B	C	D	E	F	G	H	I	J	K	L	M	N
IP	Transaction_date	Store_nbr	Product_id	Quantity_sold	Onpromotion	Selling_price	Total_sales	Month_extraction	Date_extraction	Stock_level	Reorder_level	Department	Category
0	01-01-2013	25	103665	7	No promotion	115	805	January	Tuesday	45	12	Dairy	Milk
1	01-01-2013	25	105574	1	No promotion	120	120	January	Tuesday	30	13	Bakery	Bread
2	01-01-2013	25	105575	2	No promotion	89	178	January	Tuesday	50	15	Snacks	Soft drinks
3	01-01-2013	25	108079	1	No promotion	150	150	January	Tuesday	100	20	Beverages	Rice
4	01-01-2013	25	108701	1	No promotion	120	120	January	Tuesday	34	25	Frozen Foods	Cheese
5	01-01-2013	25	108786	3	No promotion	67	201	January	Tuesday	56	40	Personal care	Biscuits
6	01-01-2013	25	108797	1	No promotion	78	78	January	Tuesday	67	30	Dairy	Milk
7	01-01-2013	25	108952	1	No promotion	90	90	January	Tuesday	78	23	Bakery	Bread
8	01-01-2013	25	111397	13	No promotion	120	1560	January	Tuesday	20	25	Snacks	Soft drinks
9	01-01-2013	25	114790	3	No promotion	1500	4500	January	Tuesday	40	67	Beverages	Rice
10	01-01-2013	25	114800	1	No promotion	89	89	January	Tuesday	30	98	Frozen Foods	Cheese
11	01-01-2013	25	115267	1	No promotion	90	90	January	Tuesday	67	20	Personal care	Biscuits
12	01-01-2013	25	115611	1	No promotion	95	95	January	Tuesday	90	10	Dairy	Milk
13	01-01-2013	25	115693	1	No promotion	85	85	January	Tuesday	100	30	Bakery	Bread
14	01-01-2013	25	115720	5	No promotion	65	325	January	Tuesday	56	20	Snacks	Soft drinks
15	01-01-2013	25	115850	1	No promotion	75	75	January	Tuesday	30	30	Beverages	Rice
16	01-01-2013	25	115891	6	No promotion	150	900	January	Tuesday	15	20	Frozen Foods	Cheese
17	01-01-2013	25	115892	10	No promotion	115	1150	January	Tuesday	20	10	Personal care	Biscuits
18	01-01-2013	25	115894	5	No promotion	115	575	January	Tuesday	10	15	Dairy	Milk
19	01-01-2013	25	119024	1	No promotion	120	120	January	Tuesday	90	17	Bakery	Bread
20	01-01-2013	25	119026	1	No promotion	234	234	January	Tuesday	40	18	Snacks	Soft drinks
21	01-01-2013	25	119141	1	No promotion	267	267	January	Tuesday	30	10	Beverages	Rice
22	01-01-2013	25	119187	1	No promotion	250	250	January	Tuesday	20	20	Frozen Foods	Cheese
23	01-01-2013	25	119624	4	No promotion	120	480	January	Tuesday	25	30	Personal care	Biscuits
24	01-01-2013	25	123927	1	No promotion	90	90	January	Tuesday	65	30	Dairy	Milk
25	01-01-2013	25	129635	11	No promotion	88	968	January	Tuesday	89	40	Bakery	Bread
26	01-01-2013	25	153239	3	No promotion	82	246	January	Tuesday	50	20	Snacks	Soft drinks
27	01-01-2013	25	153267	9	No promotion	130	1170	January	Tuesday	30	10	Beverages	Rice
28	01-01-2013	25	153395	7	No promotion	115	805	January	Tuesday	20	20	Frozen Foods	Cheese
29	01-01-2013	25	155607	2	No promotion	120	240	January	Tuesday	43	30	Personal care	Biscuits
30	01-01-2013	25	158680	1	No promotion	89	89	January	Tuesday	67	30	Dairy	Milk
31	01-01-2013	25	158788	4	No promotion	150	600	January	Tuesday	100	20	Bakery	Bread
32	01-01-2013	25	158842	9	No promotion	120	1080	January	Tuesday	34	30	Snacks	Soft drinks
33	01-01-2013	25	158875	1	No promotion	67	67	January	Tuesday	56	15	Beverages	Rice
34	01-01-2013	25	158956	5	No promotion	78	390	January	Tuesday	67	16	Frozen Foods	Shampoo
35	01-01-2013	25	159156	29.904	No promotion	90	2691.36	January	Tuesday	78	22	Personal care	Biscuits
36	01-01-2013	25	162066	2	No promotion	120	240	January	Tuesday	20	25	Dairy	Milk
Train_small													
Monthly sales		High Demand Products		Department-wise sales		Category-wise sales		Orc ... + : ◀ ▶					

J	K	L	M	N	O	P	Q	R	S	T	U
Date_extraction	Stock_level	Reorder_level	Department	Categories	Remaining_stock	Reorder_status	Price_category	Discount_Percent	Discount_Price	Cost_price	Profit
Tuesday	45	12	Dairy	Milk	38	Stock Available	High Price	10%	103.5	10	93.5
Tuesday	30	13	Bakery	Bread	29	Stock Available	High Price	10%	108	20	88
Tuesday	50	15	Snacks	Soft drinks	48	Stock Available	Normal Price	15%	75.65	10	65.65
Tuesday	100	20	Beverages	Rice	99	Stock Available	High Price	20%	120	20	100
Tuesday	34	25	Frozen Foods	oil	33	Stock Available	High Price	25%	90	5	85
Tuesday	56	40	Personal care	Biscuits	53	Stock Available	Normal Price	30%	46.9	30	16.9
Tuesday	67	30	Dairy	Milk	66	Stock Available	Normal Price	10%	70.2	20	50.2
Tuesday	78	23	Bakery	Bread	77	Stock Available	Normal Price	20%	72	40	32
Tuesday	20	25	Snacks	Soft drinks	7	Reorder needed	High Price	15%	102	10	92
Tuesday	40	67	Beverages	Rice	37	Reorder needed	High Price	5%	1425	20	1405
Tuesday	30	98	Frozen Foods	oil	29	Reorder needed	Normal Price	10%	80.1	10	70.1
Tuesday	67	20	Personal care	Biscuits	66	Stock Available	Normal Price	15%	76.5	10	66.5
Tuesday	90	10	Dairy	Milk	89	Stock Available	Normal Price	20%	76	20	56
Tuesday	100	30	Bakery	Bread	99	Stock Available	Normal Price	25%	63.75	10	53.75
Tuesday	56	20	Snacks	Soft drinks	51	Stock Available	Normal Price	10%	58.5	5	53.5
Tuesday	30	30	Beverages	Rice	29	Reorder needed	Normal Price	10%	67.5	30	37.5
Tuesday	15	20	Frozen Foods	oil	9	Reorder needed	High Price	40%	90	20	70
Tuesday	20	10	Personal care	Biscuits	10	Reorder needed	High Price	15%	97.75	10	87.75
Tuesday	10	15	Dairy	Milk	5	Reorder needed	High Price	5%	109.25	20	89.25
Tuesday	90	17	Bakery	Bread	89	Stock Available	High Price	10%	108	30	78
Tuesday	40	18	Snacks	Soft drinks	39	Stock Available	High Price	15%	198.9	40	158.9
Tuesday	30	10	Beverages	Rice	29	Stock Available	High Price	20%	213.6	40	173.6
Tuesday	20	20	Frozen Foods	oil	19	Reorder needed	High Price	25%	187.5	10	177.5
Tuesday	25	30	Personal care	Biscuits	21	Reorder needed	High Price	30%	84	20	64
Tuesday	65	30	Dairy	Milk	64	Stock Available	Normal Price	10%	81	30	51
Tuesday	89	40	Bakery	Bread	78	Stock Available	Normal Price	20%	70.4	10	60.4
Tuesday	50	20	Snacks	Soft drinks	47	Stock Available	Normal Price	15%	69.7	10	59.7
Tuesday	30	10	Beverages	Rice	21	Stock Available	High Price	5%	123.5	20	103.5
Tuesday	20	20	Frozen Foods	oil	13	Reorder needed	High Price	10%	103.5	30	73.5
Tuesday	43	30	Personal care	Biscuits	41	Stock Available	High Price	15%	102	40	62
Tuesday	67	30	Dairy	Milk	66	Stock Available	Normal Price	20%	71.2	10	61.2
Tuesday	100	20	Bakery	Bread	96	Stock Available	High Price	25%	112.5	20	92.5
Tuesday	34	30	Snacks	chips	25	Reorder needed	High Price	30%	84	10	74
Tuesday	56	15	Beverages	soap	55	Stock Available	Normal Price	10%	60.3	10	50.3
Tuesday	67	16	Frozen Foods	shampoo	62	Stock Available	Normal Price	40%	46.8	10	36.8
Tuesday	78	22	Personal care	bun	48.096	Stock Available	Normal Price	15%	76.5	10	66.5
Tuesday	20	25	Dairy	Butter	18	Reorder needed	High Price	5%	114	10	104

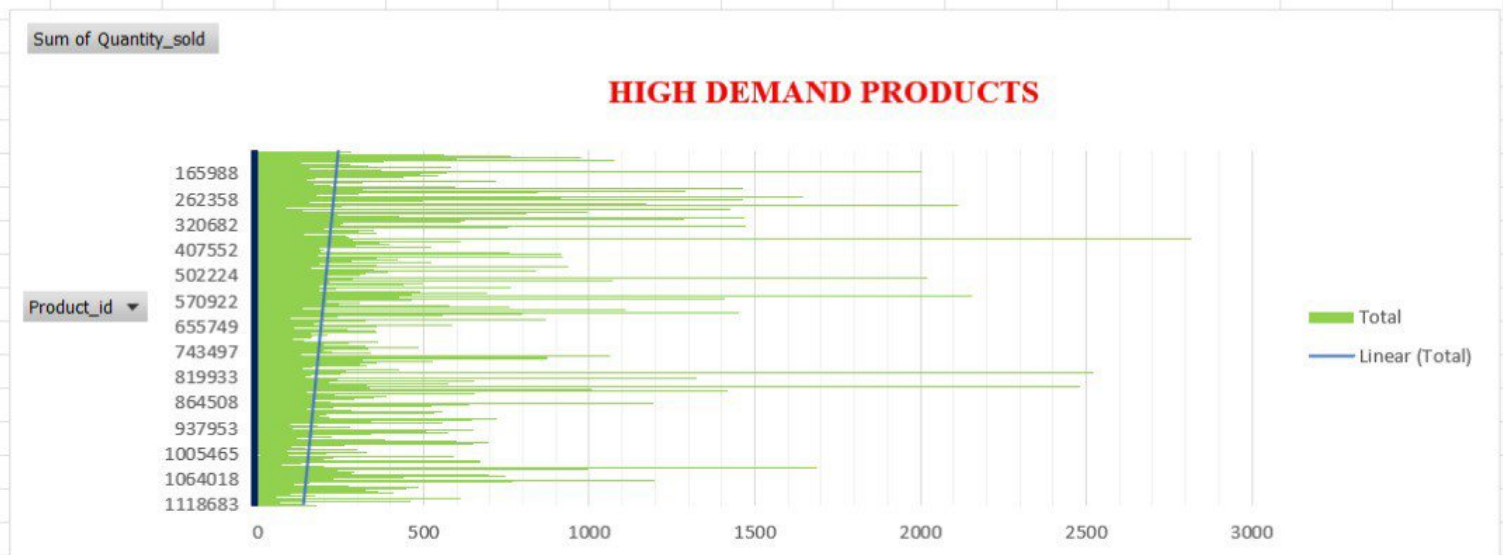
MONTHLY-WISE SALES INFORMATION



Row Labels Sum of Quantity_sold

1118683	6
1114749	176
1113872	117
1113852	83
1113847	100
1110679	4
1109391	100
1109390	67
1109389	66
1109326	81
1109325	140
1109173	8
1109116	21
1105831	2
1105299	69
1105228	40
1105215	1
1105214	106
1105213	10
1105212	461
1105211	260
1105210	83
1104725	275
1104650	1
1104599	45
1104598	142
1102976	94
1102975	27
1102971	63
1102970	53
1102268	247
1101724	144
1099990	34
1098624	401
1097187	128

HIGH DEMAND PRODUCTS VISUALIZATION



DEPARTMENT -WISE SALES INFORMATION

Row Labels	Sum of Total_sales
Bakery	8082122.872
Beverages	8283410.023
Dairy	11017268.99
Frozen Foods	7548361.544
Personal care	8379553.691
Snacks	8158523.046
Grand Total	51469240.16

Sum of Total_sales

DEPARTMENT CONTRIBUTION



Department

- Bakery
- Beverages
- Dairy
- Frozen Foods
- Personal care
- Snacks

CATEGORY-WISE SALES INFORMATION

Row Labels	Sum of Total_sales
Biscuits	3827063.275
Bread	5242555.21
bun	2738272.073
Butter	2885577.813
Cake	2325308.027
chips	2782507.712
Juice	2244207.395
Milk	5725944.714
oil	3538695.225
Rice	4233311.839
shampoo	2494042.588
soap	2705547.829
Soft drinks	4582626.479
Sugar	3267022.301
Tea	2697620.287
(blank)	178937.395
Grand Total	51469240.16



REORDER STATUS

Row Labels	Sum of Product_id
Reorder needed	6023378320
Stock Available	12275223095
Grand Total	18298601415

PRICE CATEGORY

Row Labels	Sum of Total_sales
High Price	39558946.71
Normal Price	11910293.46
Grand Total	51469240.16

DISCOUNT PRICE

Row Labels	Sum of Total_sales
39	54980.705
40.2	110406.352
45	55925.925
45.5	55048.955
46.8	71700.252
46.9	113665.433
48.75	45702.28
49.2	64591.154
50.25	117392.844
50.4	31562.58
51	47002.535
52	50520.73
52.2	52308.924
52.5	77285.175
52.8	103388.208
53.4	152021.79
53.6	114240.025
54	243619.11
54.6	95136.756
55.25	107044.86
56.25	66742.725
56.4	20981.458
56.95	234713.73
57	120502.085
57.4	99002.372
58.5	192570.04
58.8	88340.966
59.5	71015.12
60	131133.7
60.3	259639.003
60.9	55314.861
61.5	110363.882
61.6	108231.288
61.75	44658.38



SMART GROCERY STORE SALES & INVENTORY DASHBOARD



TOTAL SALES

51469240.16



TOTAL PROFIT

3421450.05



LOW STOCK

9972



TOTAL QUALITY SOLD

296695.723



TOTAL ORDERS

18298601415

On promotion

Promotion

No promotion



Department

Bakery Beverages
Dairy Frozen Fo...
Personal care Snacks

Store_nbr

1 2
3 4
5 6
7 8

Month_extraction

January

MONTHLY SALES

Sum of Total_sales

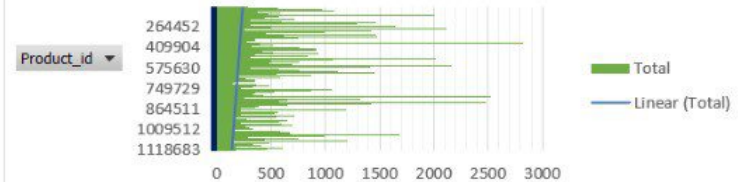
MONTHLY DEMAND FORECASTING



HIGH DEMAND

Sum of Quantity_sold

HIGH DEMAND PRODUCTS



DEPARTMENT-WISE SALES

Sum of Total_sales

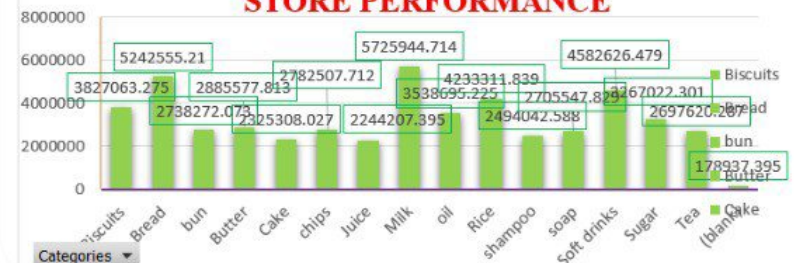
DEPARTMENT CONTRIBUTION



CATEGORY-WISE SALES

Sum of Total_sales

STORE PERFORMANCE





Navigator

SCHEMAS

Filter objects

- companydb
- companydb1
- ezhil
- new_schema
 - Tables
 - Views
 - Stored Procedures
 - Functions
- retail_project
 - Tables
 - sales_data
 - Views
 - Stored Procedures
 - Functions
- salesdb
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sql.1
- statisticsdb
- sys
- user

Administration Schemas

Information

Table: sales_data

Columns:

ID	int
Transaction_date	text
Store_nbr	int
Product_id	int
Quantity_sold	int
Onpromotion	text
Selling_price	int

sql SQL file4 SQL PRAc SQL File 7* x sales_data - Table



Limit to 1000 rows

```
1 CREATE DATABASE retail_project;
2 USE retail_project;
3 SELECT*FROM sales_data;
4 SELECT COUNT(*) FROM sales_data;
```

Result Grid Filter Rows: Export: Wrap Cell Contents:

	ID	Transaction_date	Store_nbr	Product_id	Quantity_sold	Onpromotion
▶	0	1/1/2013	25	103665	7	No promotion
	1	1/1/2013	25	105574	1	No promotion
	2	1/1/2013	25	105575	2	No promotion
	3	1/1/2013	25	108079	1	No promotion
	4	1/1/2013	25	108701	1	No promotion
	5	1/1/2013	25	108786	3	No promotion
	6	1/1/2013	25	108797	1	No promotion
	7	1/1/2013	25	108952	1	No promotion
	8	1/1/2013	25	111397	13	No promotion
	9	1/1/2013	25	114790	3	No promotion

Result Grid

Form Editor

Field Types

sales_data 3 x

Read Only

Context Help

Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
39	20:25:13	Apply changes to sales_data	No changes detected	
40	20:25:28	Apply changes to sales_data	No changes detected	
41	20:25:29	Apply changes to sales_data	No changes detected	
42	20:28:50	SELECT*FROM sales_data LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.016 sec
43	20:33:29	SELECT COUNT(*) FROM sales_data LIMIT 0. 1...	1 row(s) returned	0.031 sec / 0.000 sec

Automatic context help disabled. Use the toolbar manually get help for the current caret position or toggle automatic help.



Navigators

SCHEMAS

Filter objects

- companydb
- companydb1
- ezhil
- new_schema
 - Tables
 - Views
 - Stored Procedures
 - Functions
- retail_project**
 - Tables**
 - sales_data**
 - Views
 - Stored Procedures
 - Functions
- salesdb
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sql.1
- statisticsdb
- sys
- user

sql SQL file4 SQL PRAC SQL File 7* x sales_data - Table

Limit to 1000 rows

```
1 CREATE DATABASE retail_project;
2 USE retail_project;
3 SELECT * FROM sales_data;
4 SELECT COUNT(*) FROM sales_data;
```

Result Grid Filter Rows: Export: Wrap Cell Content:

	COUNT(*)
▶	29998

Result Grid

Form Editor

Field Types



Navigator

SCHEMAS

Filter objects

- companydb
- companydb1
- ezhil
- new_schema
 - Tables
 - Views
 - Stored Procedures
 - Functions
- retail_project
 - Tables
 - sales_data
 - Views
 - Stored Procedures
 - Functions
- salesdb
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sql.1
- statisticsdb
- sys
- user

sql

SQL file4

SQL PRAC

SQL File 7

sales_data - Table

SQLAd



Limit to 1000 rows

```
1 CREATE DATABASE retail_project;
2 USE retail_project;
3 SELECT * FROM sales_data;
4 SELECT COUNT(*) FROM sales_data;
5 SELECT * FROM sales_data WHERE quantity_sold IS NULL;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:

Result
GridForm
EditorField
Types

	ID	Transaction_date	Store_nbr	Product_id	Quantity_sold	Onpromotion
--	----	------------------	-----------	------------	---------------	-------------

File Edit View Query Database Server Tools Scripting Help



Navigator

SCHEMAS

Filter objects

- companydb
- companydb1
- ezhil
- new_schema
 - Tables
 - Views
 - Stored Procedures
 - Functions
- retail_project
 - Tables
 - sales_data
 - Views
 - Stored Procedures
 - Functions
- salesdb
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sql.1
- statisticsdb
- sys
- user

SQL QUERY

sql

SQL file4

SQL PRAC

SQL File 7*

sales_data - Table



Limit to 1000 rows

```
1 CREATE DATABASE retail_project;
2 USE retail_project;
3 SELECT * FROM sales_data;
4 SELECT COUNT(*) FROM sales_data;
5 SELECT * FROM sales_data WHERE quantity_sold IS NULL;
6 SELECT SUM(total_sales) AS total_revenue FROM sales_data;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	total_revenue
▶	51468242

File Edit View Query Database Server Tools Scripting Help



Navigators

SCHEMAS

Filter objects

- companydb
- companydb1
- ezhil
- new_schema
 - Tables
 - Views
 - Stored Procedures
 - Functions
- retail_project
 - Tables
 - sales_data
 - Views
 - Stored Procedures
 - Functions
- salesdb
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sql.1
- statisticsdb
- sys
- user

SQL QUERY

sql

SQL file4

SQL PRAc

SQL File 7

sales_data - Table



```
1 CREATE DATABASE retail_project;
2 USE retail_project;
3 SELECT * FROM sales_data;
4 SELECT COUNT(*) FROM sales_data;
5 SELECT * FROM sales_data WHERE quantity_sold IS NULL;
6 SELECT SUM(total_sales) AS total_revenue FROM sales_data;
7 SELECT AVG(total_sales) AS average_sales FROM sales_data;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	average_sales
▶	1715.7224



```
3 • SELECT * FROM sales_data;  
4 • SE  
5 • SELECT * FROM sales_data WHERE quantity_sold IS NULL;  
6 • SELECT SUM(total_sales) AS total_revenue FROM sales_data;  
7 • SELECT AVG(total_sales) AS average_sales FROM sales_data;  
8 • SELECT product_id, SUM(quantity_sold) AS total_quantity FROM sales_data  
9 GROUP BY product_id ORDER BY total_quantity DESC LIMIT 10;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



Fetch rows:



	product_id	total_quantity
▶	364606	2819
	807493	2524
	839362	2481
	559870	2156
	265559	2113
	502331	2022
	159156	2004
	364738	1757
	1047679	1688
	257847	1644

```

6 • SELECT SUM(total_sales) AS total_revenue FROM sales_data;
7 • SELECT AVG(total_sales) AS average_sales FROM sales_data;
8 • SELECT product_id, SUM(quantity_sold) AS total_quantity FROM sales_data
9   GROUP BY product_id ORDER BY total_quantity DESC LIMIT 10;
10 • SELECT product_id, SUM(quantity_sold) AS total_quantity FROM sales_data
11   GROUP BY product_id ORDER BY total_quantity ASC LIMIT 10;
12

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content:  | Fetch rows: 

	product_id	total_quantity
▶	1091338	0
	789907	1
	915822	1
	883336	1
	1005465	1
	1104650	1
	1009946	1
	1080009	1
	1005464	1
	1105215	1



```
6 • SELECT SUM(total_sales) AS total_revenue FROM sales_data;
7 • SELECT AVG(total_sales) AS average_sales FROM sales_data;
8 • SELECT product_id, SUM(quantity_sold) AS total_quantity FROM sales_data
9   GROUP BY product_id ORDER BY total_quantity DESC LIMIT 10;
10 • SELECT product_id, SUM(quantity_sold) AS total_quantity FROM sales_data
11   GROUP BY product_id ORDER BY total_quantity ASC LIMIT 10;
12 • SELECT store_nbr, SUM(total_sales) AS sales FROM sales_data GROUP BY store_nbr;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	store_nbr	sales
▶	25	1278191
	1	1344324
	2	1745107
	3	4204842
	4	1898802
	5	1767216
	6	2481560
	7	2309082
	8	2525380
	9	2624545
	10	849236
	11	2802651



```
7 • SELECT AVG(total_sales) AS average_sales FROM sales_data;
8 • SELECT product_id, SUM(quantity_sold) AS total_quantity FROM sales_data
9   GROUP BY product_id ORDER BY total_quantity DESC LIMIT 10;
10 • SELECT product_id, SUM(quantity_sold) AS total_quantity FROM sales_data
11   GROUP BY product_id ORDER BY total_quantity ASC LIMIT 10;
12 • SELECT store_nbr, SUM(total_sales) AS sales FROM sales_data GROUP BY store_nbr;
13 • SELECT Transaction_date, SUM(total_sales) FROM sales_data GROUP BY Transaction_date;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	Transaction_date	SUM(total_sales)
▶	1/1/2013	384626
	2/1/2013	51083616



```
8 • SELECT product_id, SUM(quantity_sold) AS total_quantity FROM sales_data
9   GROUP BY product_id ORDER BY total_quantity DESC LIMIT 10;
10 • SELECT product_id, SUM(quantity_sold) AS total_quantity FROM sales_data
11   GROUP BY product_id ORDER BY total_quantity ASC LIMIT 10;
12 • SELECT store_nbr, SUM(total_sales) AS sales FROM sales_data GROUP BY store_nbr;
13 • SELECT Transaction_date, SUM(total_sales) FROM sales_data GROUP BY Transaction_date;
14 • SELECT product_id, SUM(quantity_sold) AS demand FROM sales_data GROUP BY product_id ORDER BY demand DESC LIMIT 5;
```

Result Grid Filter Rows: Export: Wrap Cell Content: Fetch rows:

	product_id	demand
▶	364606	2819
	807493	2524
	839362	2481
	559870	2156
	265559	2113



Limit to 1000 rows

```
11 GROUP BY product_id ORDER BY total_quantity ASC LIMIT 10;
12 • SELECT store_nbr, SUM(total_sales) AS sales FROM sales_data GROUP BY store_nbr;
13 • SELECT Transaction_date, SUM(total_sales) FROM sales_data GROUP BY Transaction_date;
14 • SELECT product_id, SUM(quantity_sold) AS demand FROM sales_data GROUP BY product_id ORDER BY demand DESC LIMIT 5;
15 • SELECT * FROM sales_data
16 WHERE Stock_level < 'Reorder level';
17 • SELECT product_id, stock_level, 'Reorder level' FROM sales_data WHERE Reorder_status = 'Reorder Needed';
```

Result Grid Filter Rows:

Export: Wrap Cell Content: Fetch rows:

	product_id	stock_level	Reorder level
▶	111397	20	Reorder level
	114790	40	Reorder level
	114800	30	Reorder level
	115850	30	Reorder level
	115891	15	Reorder level
	115892	20	Reorder level
	115894	10	Reorder level
	119187	20	Reorder level
	119624	25	Reorder level
	153395	20	Reorder level
	158842	34	Reorder level



```
12 • SELECT store_nbr, SUM(total_sales) AS sales FROM sales_data GROUP BY store_nbr;
13 • SELECT Transaction_date, SUM(total_sales) FROM sales_data GROUP BY Transaction_date;
14 • SELECT product_id, SUM(quantity_sold) AS demand FROM sales_data GROUP BY product_id ORDER BY demand DESC LIMIT 10;
15 • SELECT * FROM sales_data
16 WHERE Stock_level < 'Reorder level';
17 • SELECT product_id, stock_level, 'Reorder level' FROM sales_data WHERE Reorder_status = 'Reorder Needed';
18 • SELECT product_id, SUM(profit) AS total_profit FROM sales_data GROUP BY product_id ORDER BY total_profit DESC;
```

Result Grid Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:



	product_id	total_profit
▶	262991	8553.45
	115892	7747
	378330	7602.75
	265279	7382.1
	153267	7165.75
	567750	7013.35
	621300	6863.85
	1036689	6686.199999999999
	311954	6674.749999999999
	598414	6669.9
	220613	6614.550000000001
	323921	6528.500000000001

13 • `SELECT Transaction_date, SUM(total_sales) FROM sales_data GROUP BY Transaction_date;`
 14 • `SELECT product_id, SUM(demand) FROM sales_data GROUP BY product_id ORDER BY demand DESC LIMIT 5;`
 15 • `SELECT * FROM sales_data`
 16 `WHERE Stock_level < 'Reorder level';`
 17 • `SELECT product_id, stock_level, 'Reorder level' FROM sales_data WHERE Reorder_status = 'Reorder Needed';`
 18 • `SELECT product_id, SUM(profit) AS total_profit FROM sales_data GROUP BY product_id ORDER BY total_profit DESC;`
 19 • `SELECT AVG(discount_percent) AS avg_discount FROM sales_data;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: ☐

	avg_discount
▶	18.961430762050803



Limit to 1000 rows

```
14 • SELECT product_id, SUM(quantity_sold) AS demand FROM sales_data GROUP BY product_id ORDER BY demand DESC LIMIT 5;
15 • S Execute the selected portion of the script or everything, if there is no selection
16 WHERE Stock_level < 'Reorder level';
17 • SELECT product_id, stock_level, 'Reorder level' FROM sales_data WHERE Reorder_status = 'Reorder Needed';
18 • SELECT product_id, SUM(profit) AS total_profit FROM sales_data GROUP BY product_id ORDER BY total_profit DESC;
19 • SELECT AVG(discount_percent) AS avg_discount FROM sales_data;
20 • SELECT product_id, discount_percent FROM sales_data ORDER BY discount_percent DESC LIMIT 10;
```

Result Grid Filter Rows: Export: Wrap Cell Content: Fetch rows:

	product_id	discount_percent
▶	796394	5%
	315474	5%
	838407	5%
	933780	5%
	311994	5%
	699745	5%
	949299	5%
	457688	5%
	876663	5%
	813770	5%



```
16 WHERE Stock_level < 'Reorder level';
17 • SELECT product_id, stock_level, 'Reorder level' FROM sales_data WHERE Reorder_status = 'Re
18 • SELECT product_id, SUM(profit) AS total_profit FROM sales_data GROUP BY product_id ORDER
19 • SELECT AVG(discount_percent) AS avg_discount FROM sales_data;
20 • SELECT product_id, discount_percent FROM sales_data ORDER BY discount_percent DESC LIMIT
21 • SELECT MONTH(transaction_date) AS month_no, SUM(total_sales) AS monthly_sales
22 FROM sales_data GROUP BY MONTH(transaction_date);
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	month_no	monthly_sales
▶	NULL	51468242

```
21 • SELECT MONTH(transaction_date) AS month_no, SUM(total_sales) AS monthly_sales
22 FROM sales_data GROUP BY MONTH(transaction_date);
23 • SELECT product_id, remaining_stock FROM sales_data ORDER BY remaining_stock asc;
```

Result Grid |   Filter Rows:

Export: 

Wrap Cell Content: 

Fetch rows: 

	product_id	remaining_stock
▶	584126	-509
	757888	-475
	757889	-426
	1047679	-393
	257847	-385
	364738	-272
	807493	-257
	258376	-257
	1047679	-231
	159156	-223
	219150	-209
	903283	-195

-475



```
18 • SELECT product_id, SUM(profit) AS total_profit FROM sales_data GROUP BY product_id ORDER BY total_
19 • SELECT AVG(discount_percent) AS avg_discount FROM sales_data;
20 • SELECT product_id, discount_percent FROM sales_data ORDER BY discount_percent DESC LIMIT 10;
21 • SELECT MONTH(transaction_date) AS month_no, SUM(total_sales) AS monthly_sales
22 FROM sales_data GROUP BY MONTH(transaction_date);
23 • SELECT product_id, remaining_stock FROM sales_data ORDER BY remaining_stock asc;
24 • SELECT onpromotion, SUM(total_sales) FROM sales_data GROUP BY onpromotion;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	onpromotion	SUM(total_sales)
▶	No promotion	51347416
	Promotion	120826

25 • `CREATE VIEW sales_summary AS SELECT product_id, SUM(total_sales)AS sales FROM sales_data GROUP BY product_id;`

Output

Action Output			
#	Time	Action	Message
71	21:44:14	SELECT product_id, discount_percent FROM sales_data ORDER BY discount_percent DESC LIMIT 10	10 row(s) returned
72	21:45:52	SELECT MONTH(transaction_date) AS month_no, SUM(total_sales) AS monthly_sales FROM sales_data GR...	1 row(s) returned
73	21:47:07	ELECT product_id, Remaining_stock FROM sales_data ORDER BY Remaining_stock asc	Error Code: 1064. You have an error in yo
74	21:47:33	SELECT product_id, remaining_stock FROM sales_data ORDER BY remaining_stock asc LIMIT 0, 1000	1000 row(s) returned
75	21:48:23	SELECT onpromotion, SUM(total_sales) FROM sales_data GROUP BY onpromotion LIMIT 0, 1000	2 row(s) returned
76	21:49:58	CREATE VIEW sales_summary AS SELECT product_id, SUM(total_sales)AS sales FROM sales_data GROUP ...	0 row(s) affected



```
25 • CREATE VIEW sales_summary AS SELECT product_id, SUM(total_sales) AS sales FROM sales_data GROUP BY product_id;
26 • SE Execute the selected portion of the script or everything, if there is no selection
27 • SELECT product_id,
28     SUM(quantity_sold) OVER(PARTITION BY product_id) AS running_sales
29     FROM sales_data;
30 • SELECT product_id, SUM(total_sales) AS sales, RANK() OVER(ORDER BY SUM(total_sales) DESC) AS product_rank FROM
31     sales_data GROUP BY product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	product_id	sales	product_rank
▶	265559	519235	1
	220435	511297	2
	273528	361764	3
	364606	360598	4
	315176	351267	5
	839362	350991	6
	314384	350905	7
	364738	324886	8
	862454	315764	9
	559870	309166	10
	320682	305209	11
	807493	288082	12

Result 24 x

```

21 • SELECT MONTH(transaction date) AS month no, SUM(total sales) AS monthly_sales
22 FROM sales_data;
23 • SELECT product_id, remaining_stock FROM sales_data ORDER BY remaining_stock asc;
24 • SELECT onpromotion, SUM(total_sales) FROM sales_data GROUP BY onpromotion;
25 • CREATE VIEW sales_summary AS SELECT product_id, SUM(total_sales) AS sales FROM sales_data;
26 • SELECT * FROM sales_summary;
27 • SELECT product_id,
28 SUM(quantity sold) OVER(PARTITION BY product_id) AS running sales
    
```

	product_id	sales
▶	103665	13181
	105574	26932
	105575	73151
	108079	2924
	108701	17123
	108786	50245
	108797	19115
	108952	10163
	111397	26839
	114790	89545
	114800	30387
	115267	62050



```
27 • SELECT product_id,  
28 SU Execute the selected portion of the script or everything, if there is no selection les  
29 FROM sales_data;  
30 • SELECT product_id, SUM(total_sales) AS sales, RANK() OVER(ORDER BY SUM(total_sales) DESC) AS product_r  
31 sales_data GROUP BY product_id;  
32 • SELECT Transaction_date,AVG(quantity_sold) AS avg_demand  
33 FROM sales_data GROUP BY Transaction_date;
```

Result Grid Filter Rows: Export: Wrap Cell Content:

	Transaction_date	avg_demand
▶	1/1/2013	4.3646
	2/1/2013	9.9987

```
32 • SELECT Transaction_date,AVG(quantity_sold) AS avg_demand
33 FROM sales_data GROUP BY Transaction_date;
```

```
38 • SELECT * FROM items;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	item_nbr	family	class	perishable
▶	96995	GROCERY I	1093	0
	99197	GROCERY I	1067	0
	103501	CLEANING	3008	0
	103520	GROCERY I	1028	0
	103665	BREAD/BAKERY	2712	1
	105574	GROCERY I	1045	0
	105575	GROCERY I	1045	0
	105576	GROCERY I	1045	0
	105577	GROCERY I	1045	0
	105693	GROCERY I	1034	0
	105737	GROCERY I	1044	0
	105857	GROCERY I	1092	0

```
37
38 • SELECT * FROM items;
39 • SELECT COUNT(*) FROM items;
40
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	COUNT(*)
▶	4100

27

```
38 • SELECT * FROM items;  
39 • SELECT COUNT(*) FROM items;  
40 • SELECT family, COUNT(*) AS total_products FROM items GROUP BY family;  
41
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

family	total_products
GROCERY I	1334
CLEANING	446
BREAD/BAKERY	134
DELI	91
POULTRY	54
EGGS	41
PERSONAL CARE	153
LINGERIE	20
BEVERAGES	613
AUTOMOTIVE	20
DAIRY	242
GROCERY II	14

```

38 • SELECT * FROM items;
39 • SELECT COUNT(*) FROM items;
40 • SELECT family, COUNT(*) AS total_products FROM items GROUP BY family;
41 • SELECT * FROM items WHERE perishable = 1;

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

item_nbr	family	class	perishable
103665	BREAD/BAKERY	2712	1
108696	DELI	2636	1
108698	DELI	2644	1
108701	DELI	2644	1
108831	POULTRY	2416	1
108833	EGGS	2502	1
122095	DAIRY	2124	1
123347	DAIRY	2114	1
127547	EGGS	2502	1
129635	DAIRY	2112	1
153239	BREAD/BAKERY	2712	1
153395	BREAD/BAKERY	2704	1

```

38 • SELECT * FROM items;
39 • SELECT COUNT(*) FROM items;
40 • SELECT family, COUNT(*) AS total_products FROM items GROUP BY family;
41 • SELECT * FROM items WHERE perishable = 1;
42 • SELECT * FROM items WHERE perishable = 0;
43

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content:  | Fetch rows: 

	item_nbr	family	class	perishable
▶	96995	GROCERY I	1093	0
	99197	GROCERY I	1067	0
	103501	CLEANING	3008	0
	103520	GROCERY I	1028	0
	105574	GROCERY I	1045	0
	105575	GROCERY I	1045	1045
	105576	GROCERY I	1045	0
	105577	GROCERY I	1045	0
	105693	GROCERY I	1034	0
	105737	GROCERY I	1044	0
	105857	GROCERY I	1092	0



37

38 • **SEL** Execute the selected portion of the script or everything, if there is no selection39 • **SELECT** COUNT(*) **FROM** items;40 • **SELECT** family, COUNT(*) **AS** total_products **FROM** items **GROUP BY** family;41 • **SELECT** * **FROM** items **WHERE** perishable = 1;42 • **SELECT** * **FROM** items **WHERE** perishable = 0;43 • **SELECT** class, COUNT(*) **AS** total_items **FROM** items **GROUP BY** class;44 • **SELECT** c.item_nbr, i.family, SUM(c.unit_sales) **AS** total_sales **FROM** sales_data c

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	class	total_items
▶	1093	5
	1067	2
	3008	26
	1028	48
	2712	17
	1045	24
	1034	98
	1044	30
	1092	19
	1032	65
	1030	51
	1075	2

```
38 • SELECT * FROM items;
39 • SELECT COUNT(*) FROM items;
40 • SELECT family, COUNT(*) AS total_products FROM items GROUP BY family;
41 • SELECT * FROM items WHERE perishable = 1;
42 • SELECT * FROM items WHERE perishable = 0;
43 • SELECT class, COUNT(*) AS total_items FROM items GROUP BY class;
44 • SELECT s.product_id, i.family, SUM(s.quantity_sold) AS total_sales
45 ON s.product_id = i.item_nbr GROUP BY i.family;
```